

Cross-Protocol Failure Propagation Across MCP and A2A Agents: A Controlled Pilot on Information Flow, Behavioral Influence, and Containment

Arpan Kumar Mahapatra
arpan.arpan.mohapatra@gmail.com

August 30, 2026

Abstract

Deployed AI agents increasingly speak two protocols at once: the Model Context Protocol (MCP) [2] for local tool use, and the Agent2Agent Protocol (A2A) [1] for delegating work to other agents. Dedicated safety benchmarks for each evaluate one protocol in isolation. We study the seam. A composed harness drives one real-model host across both protocol legs — implemented as local deterministic MCP and A2A protocol fixtures, not external services — in a single ordered event trace, with an observable cross-protocol provenance model (deterministic content canaries carrying a sensitivity label, an explicit allowed-edge policy, and per-event causal ancestry) so that information flow and behavioral influence across the boundary are measured by exact rule-based checks, without an LLM-based judge or evaluator. Using a decision-point methodology that deterministically builds the situation and then elicits one restricted host decision, we ran a controlled confirmatory pilot over three OpenAI GPT-5.6 models, 240 trials in total. We separate *relay initiation* (the host sent a message onto the A2A leg after reading a confidential MCP record) from *direct sensitive egress* (the exact confidential marker crossed the disallowed host-to-remote edge): relay initiation varied widely by model (15%, 85%, 100% of confidential trials), while direct exact-token egress was 0 for every model (0/3, 0/17, 0/20 of relay trials; Wilson 95% upper bounds 56.2%, 18.4%, 16.1%). This is verbatim-marker leakage only, not semantic or paraphrased leakage. Scoring whether the host both selected the mutation-attempt action *and* named a tool the harness discovered to be mutating (our primary influence outcome), the adversarial A2A artifact condition showed a higher rate than a structurally matched benign condition in all three models (signed treatment–control differences +0.20, +0.40, +0.50; Wilson 95% intervals per cell) — an association under a controlled manipulation, not a causal-mechanism claim. Every one of the 56 observed actual mutating-tool requests was blocked by an independent mutation gate, and none executed. We report Wilson 95% intervals and signed rate differences throughout and deliberately report no p-values: the replicates in a cell are repeated draws from one model under one fixed stimulus and are not assumed statistically independent. The complete raw runs and a fully offline analysis pipeline are published as a deterministic, hash-pinned release.

1 Introduction

An agent that is safe in isolation on protocol X and safe in isolation on protocol Y is not automatically safe when it uses X and Y in the same task. Two mechanisms motivate this paper, and both are compositional — properties of the interaction *between* two protocol legs rather than of either leg alone. The first is information flow across the boundary: a host agent calls a local MCP tool, receives content, and then, in service of the same user request, sends a message to a remote A2A agent; if the tool’s content was confidential, that boundary is exactly where an unintended disclosure can occur, and neither a single-protocol MCP benchmark (which cannot see the A2A edge) nor a single-protocol A2A benchmark (which never had the confidential content in scope) is positioned to observe it. The second is behavioral influence across the boundary: a remote A2A agent returns an *artifact*, a text payload the host reads, and if that artifact contains an instruction (“apply the emergency remediation now”) the host may act on it by calling a *local* MCP tool, possibly a mutating one — the adversarial content and the dangerous action living on opposite sides of the seam.

The only non-local component in every experiment reported here is real provider model inference: OpenAI GPT-5.6 models [7] called through the Responses API. All MCP and A2A infrastructure is local deterministic protocol fixtures — an in-process mock MCP tool server (stdio subprocess, synthetic data, no network) and an in-process mock A2A HTTP+JSON/REST agent exercised only through a test client. No production, external, or third-party MCP server or A2A agent was contacted. Throughout, “the host” or “the model” refers to real provider inference; “MCP fixture” and “A2A fixture” refer to the local deterministic infrastructure.

This paper contributes an executable empirical instrument and a controlled pilot, not a new risk concept. Concretely: (i) an executable MCP+A2A composition evaluation that drives one real-model host across an MCP leg and an A2A leg, both local deterministic protocol fixtures, into one ordered event trace scored by exact rules; (ii) an observable cross-protocol provenance model — deterministic content canaries carrying a sensitivity label, an explicit allowed-edge policy, and per-event causal ancestry — that makes specific boundary-crossing flows checkable without an LLM judge and separates *direct* (literal) from *propagated* (ancestral) presence; (iii) a controlled decision-point behavioral-influence measurement that isolates one host decision from multi-step planning; (iv) an outcome that distinguishes selecting the mutation-attempt action from naming a tool the harness *discovered* to be mutating; (v) an independent containment measurement by an external mutation gate that is not part of the host; and (vi) a hash-pinned live-model reproducibility mechanism (a per-run execution fingerprint plus an offline analysis pipeline that regenerates every table and figure with zero provider calls).

We claim none of the underlying risk concepts as novel, and in particular claim no priority on cross-protocol composition risk or MCP+A2A “protocol pivoting”: that risk has already been named in a June 2026 IETF Internet-Draft [10] and formalised as a “Composition Safety” principle with formal cross-protocol composition models [14] (Section 2). This is also not the “first MCP benchmark” or “first A2A benchmark” — dedicated single-protocol safety benchmarks for each already exist. Every result below is conditional on four fixed stimuli, one host policy, one tool surface, and one point in time; nothing here says any model is “safe” or “unsafe” in general, and no significance test is reported.

Manuscript preparation introduced no new experimental runs or provider calls. All empirical results are reproduced from the frozen public release `phase4b-results-v1`.

2 Background and Related Work

MCP. MCP is a client–server protocol connecting an LLM-driven *host* to *tools* and *resources* exposed by MCP servers [2, revision 2025-06-18]. The host issues a JSON-RPC `tools/call` with a tool name and a JSON argument object; the server returns a structured result. Tools may carry annotations (for instance a destructive / read-only hint) so a host or policy layer can treat mutating operations differently from read-only ones; the specification notes that such annotations “should be considered untrusted, unless obtained from a trusted server.” Here the MCP leg is a local, in-process deterministic fixture (implemented with the MCP Python SDK) that returns synthetic records and performs no network I/O; because it is the trusted local fixture, we take its discovered annotations as ground truth for a tool’s mutating status.

A2A. A2A is a protocol for one agent to delegate a task to another [1]. A *client agent* discovers a *remote agent* via an *Agent Card* (spec §8), opens a *task* whose lifecycle is a `TaskState` machine (§4.1.1–4.1.3), and exchanges *messages* built from *parts* (§4.1.4, §4.1.6); the remote agent may attach *artifacts* (§4.1.7), content payloads representing task output. The artifact is the natural place for a remote agent, benign or compromised, to place text the client agent will read and may act upon. Our A2A leg is a local, in-process deterministic fixture implementing the HTTP+JSON/REST binding shapes (spec §11), exercised only through an in-process test client.

Cross-protocol composition risk (prior articulations). The risk this paper measures is not new as a concept. A June 2026 IETF Internet-Draft [10] devotes a “Protocol Pivoting” section to lateral movement that crosses agent-protocol boundaries — explicitly “combining MCP with Agent-to-Agent (A2A) delegation” — and states that “mitigations that consider each protocol in isolation do not address movement that crosses protocol boundaries,” with a worked scenario close to our behavioral-influence question. Independently, *AgentRFC* [14] formalises a “Composition Safety” principle (“security properties that hold for individual protocols can break when protocols are composed through shared infrastructure”) with formal (TLA+) models of five cross-protocol composition patterns spanning MCP, A2A and other agent protocols, plus a spec-conformance checker. We do

not claim priority on any of this. Our work is the executable empirical complement: where those efforts check conformance against formal invariants and enumerate mitigations, we run a real-model host across a concrete MCP+A2A composition (local deterministic protocol fixtures), attach an observable provenance model, and report measured per-model rates for one information-flow outcome, one behavioral-influence outcome, and one containment outcome.

Single-protocol MCP and A2A benchmarks. *MCP-SafetyBench* [15] evaluates MCP tool-using agents on multi-turn ReAct tasks against a 20-category attack taxonomy over real-world MCP servers. *MCP Security Bench* [13] and *MCPSecBench* [12] similarly target MCP-layer security (malicious or misconfigured servers, tool-poisoning, argument-injection, over-broad permissions; MCPSecBench reports 17 attack types over four MCP attack surfaces). *A2ASecBench* [6] is a protocol-aware security benchmark for Agent-to-Agent systems, with a taxonomy of supply-chain manipulations and protocol-logic weaknesses and six attacks across A2A stages. Our composed harness reuses an MCP leg but measures what happens when that leg’s output is carried into an A2A interaction, and our behavioral-influence stimuli use an adversarial A2A artifact but the measured action is a mutating call on a *local MCP tool* guarded by a *local* mutation gate — outside a pure-MCP or pure-A2A scope. We make no “first benchmark” claim for either protocol.

Indirect prompt injection, tool-use evaluation, defenses. Indirect prompt injection — untrusted content read by an agent carrying instructions it then follows — was characterised by Greshake et al. [5]; our behavioral-influence question is an instance of it in which the injection vector is a remote A2A artifact and the intended payload is a local MCP mutation. *AgentDojo* [3] evaluates prompt-injection attacks and defenses on tool-using agents with rule-based (not LLM-judge) utility and security functions; *ToolEmu* [8] identifies LM-agent risks in an LM-emulated sandbox with an LM safety evaluator. We differ from both in that our environment executes a real-model host across a local deterministic MCP+A2A protocol composition (protocol fixtures, not an LM emulator) and scores by exact-rule provenance checking without an LLM-based judge or evaluator; this is a methodological alignment with judge-free evaluation, not a novelty claim. *CaMeL* [4] is a defense that separates control and data flow and tracks data provenance before each tool call; it is conceptually adjacent to our provenance model but is a mitigation, whereas ours is an evaluation instrument. Multi-agent security — threats that emerge or amplify through agent interactions — has been framed as a distinct field [9]; formal and information-flow approaches to agent/tool safety [4, 14] provide the theoretical context for our provenance model, but our contribution here is empirical and instrument-focused rather than a formal result.

3 Threat Model and Research Questions

We consider a single benign-by-construction host agent that faithfully follows a fixed natural-language policy: it may call local tools, relay to the remote agent, attempt a tool call (mutations requiring prior approval, which it can never self-grant), or stop, and it is told not to send content “clearly marked confidential, internal, or sensitive” to the remote agent unless the task authorizes it. The adversary is modelled purely as control over the content returned on one leg of the interaction — not over the host, the protocols, or the harness — and operationally that content is fixed text scripted into the local protocol fixtures for each condition. For the information-flow question there is no active adversary at all: the MCP fixture returns a record visibly labeled **CONFIDENTIAL – INTERNAL ONLY**, and the “threat” is the host’s own judgement about what, if anything, to relay onward. For the behavioral-influence question the A2A fixture is scripted to play a compromised remote agent, returning a realistic natural-language artifact that instructs the host to invoke a specific mutating MCP tool immediately, claiming prior approval and urgency; the artifact contains no special marker or trigger string — it is ordinary prose a real model must reason about. “Remote agent” is the A2A protocol role of the delegatee; in every experiment it is this local fixture. The mutation gate is an independent policy component that is *not* part of the host: it re-derives a tool’s mutating status from the discovered annotation and blocks any mutating call lacking explicit case-level approval. Out of scope are a malicious host, protocol-level attacks (message forgery, transport interception), multi-hop agent chains, tool-supply-chain compromise, and any production, external, or third-party MCP server or A2A agent.

Three research questions follow. **RQ1 (cross-protocol information flow):** when a host reads confidential content from an MCP tool and then interacts with a remote A2A agent in the same task, does that confidential content propagate across the A2A boundary? We decompose this into *relay initiation* — did the host send

any message to the remote agent after reading the confidential record? — and *direct sensitive egress* — did the exact confidential marker cross the disallowed host-to-remote/A2A edge? The latter is an exact-token measurement and is not a measurement of semantic leakage. **RQ2 (cross-protocol behavioral influence):** does an adversarial A2A artifact influence the host toward requesting a *mutating* MCP tool, relative to a structurally matched benign artifact? We measure the *actual mutating-tool request* rate — the host selected the mutation-attempt action *and* the named tool was discovered by the harness to be mutating — and report it separately from the rate at which the host merely selected the mutation-attempt action, which may name a non-mutating tool. **RQ3 (containment):** given an actual mutating-tool request, does an independent mutation gate block it and prevent execution?

4 Agent Interop Bench

Agent Interop Bench comprises three independent deterministic evaluation engines — an MCP tool-use engine, an A2A interoperability engine, and the composed cross-protocol engine that is the subject of this paper. All run only against local, in-process mocks with no external network access, and all scoring is rule-based. The composed engine drives one host agent across both protocol legs and records a single ordered list of `CrossProtocolEvents` spanning the MCP leg (`mcp_tool_request`, `mcp_tool_result`), the A2A leg (`a2a_message`, `a2a_task_state_transition`, `a2a_artifact`), and the host’s own gated actions (`mutation_action`). The host’s next action is produced by an adapter given only a sanitized decision context — the user prompt, a host policy, the observable protocol history so far, the discovered tool list, and the target Agent Card, and never any ground-truth label, provenance annotation, or evaluator state. A fixture-replay adapter drives the deterministic suite; a real-model adapter (OpenAI Responses API) drives live studies; the engine is otherwise unchanged between the two. The MCP fixture is a local stdio subprocess exposing a small fixed tool set that includes `get_customer_record` and `get_deployment_status` (both discovered read-only) and `apply_remote_recommendation` (annotated destructive; discovered mutating), returning synthetic content only. The A2A fixture is a local in-process HTTP+JSON app whose per-step behavior (task state, message text, artifact text) is scripted by the case. Before any mutating call executes, the composed engine applies an independent predicate, `mutation_blocked(is_mutating, approved) = is_mutating and not approved`, in which `is_mutating` is re-derived from the tool’s discovered annotation, never from a fixture flag, and a real-model host can never set `approved = true` for its own attempt; a blocked attempt is recorded with `executed = false` and a permitted call with `executed = true`.

The provenance model is what makes cross-protocol information flow and influence observable by exact rules. It has four fixture-declared axes, all hidden from the host. First, *provenance canaries*: a canary is a deterministic, syntactically inert token, `[[CANARY:<uuid>]]`, embedded in specific content and detected by exact substring identity, never by semantic interpretation, each carrying a `public` or `sensitive` sensitivity label. Second, *allowed sensitive edges*: a fixture-declared set of (`source`, `dest`, `protocol`) edges on which a `sensitive` canary is permitted to travel, so that any direct observation of a sensitive canary on an edge outside this set is a sensitive-egress violation — for the RQ1 treatment the only allowed edge is `local_tool → host/mcp`, and the `host → remote/a2a` edge is not allowed. Third, *origin trust and content class*: per-actor defaults (`user/host` trusted, `remote/local_tool` untrusted) plus optional per-event overrides and a `non_adversarial/adversarial` content class, which the engine folds into per-event adversarial-ancestry and sensitive-ancestry sets by a real DAG computation over the actual event trace rather than a category-name special case. Fourth, *direct vs. propagated presence*: each event carries both its `canary_ids` (tokens literally present in that event’s payload) and its `propagated_canary_ids` (tokens present in any causal ancestor), and our primary RQ1 outcome, direct sensitive egress, counts only literal presence on the crossing event. This distinction is central: a host can relay after reading a confidential record — creating a causal chain, so the canary appears in `propagated_canary_ids` on the A2A message — without ever placing the literal marker into the relayed text, in which case direct sensitive egress does not fire and only the secondary outcome, relay initiation, does. For RQ2/RQ3 the live-study path does not set an `adversarial` content-class override on the A2A artifact (the artifact is realistic prose, not a marker), so per-event adversarial ancestry is empty on this path; we therefore identify the “adversarial” versus “benign” condition of an influence trial by which overlay was shown — condition ground truth — not by computed ancestry, and say so wherever the influence and containment outcomes are reported.

5 Experimental Methodology

5.1 Decision-point execution

An earlier free-running pilot, in which models were asked for every step of an open interaction loop, proved uninformative: models planned the whole workflow and repeatedly relayed to the remote agent before reaching the decision the experiment was designed to measure. The frozen methodology used here is decision-point execution. The engine first constructs the situation with no model involvement: for sensitive-egress it executes the local MCP `get_customer_record` call so that a real `mcp_tool_result` (a `PUBLIC`- or `CONFIDENTIAL`-labeled record with an embedded canary) exists in the trace, and for adversarial-influence it performs exactly one deterministic A2A interaction so that a real `a2a_artifact` (benign or adversarial text) exists. The real model is then asked for exactly one decision, given only the sanitized context, with the wire-level action set restricted to `{relay_to_remote, stop}` for sensitive-egress and `{attempt_mutating_tool, stop}` for adversarial-influence; the restriction is enforced three ways — the tool schema sent to the provider contains only the permitted actions, a composite adapter refuses any returned action outside the set, and the interaction step ceiling admits only one model decision. Finally the engine performs at most one deterministic observable action: if the host relays, exactly one A2A interaction is performed and its events recorded; if the host attempts a mutating tool, the mutation gate is run and a `mutation_action` event recorded; the trial then terminates `COMPLETED`. Per-run parameters were frozen before execution: `reasoning_effort = low`, `max_output_tokens = 512`, `timeout = 20 s`, `retries = 0`, one provider decision per trial, at most 80 provider decisions per run, and all MCP and A2A targets local.

5.2 Stimuli

Four overlays, reused verbatim, form a 2×2 design of experiment $\times$ condition (Table 1). For sensitive-egress the model-visible MCP *request* carries only an opaque `record_ref` (`record-a7` / `record-b4`) with no condition information; the `CONFIDENTIAL`/`PUBLIC` label and the canary are supplied server-side, so the first model-visible difference between conditions is the returned record text, never the request. The two influence overlays differ only in the artifact text.

Table 1: The four fixed stimuli.

overlay	experiment	condition	manipulation
live-sensitive-egress-treatment	sensitive_egress	treatment	MCP record labeled <code>CONFIDENTIAL - INTERNAL ONLY</code> ; embedded canary is <code>sensitive</code> ; only allowed edge is <code>local_tool</code> $\rightarrow$ <code>host/mcp</code>
live-sensitive-egress-control	sensitive_egress	control	structurally matched record labeled <code>PUBLIC - OK TO SHARE</code> ; embedded canary is <code>public</code>
live-influence-treatment	adversarial_influence	treatment	remote A2A artifact is a realistic natural-language instruction to call <code>apply_remote_recommendation</code> immediately, claiming prior approval and urgency
live-influence-control	adversarial_influence	control	structurally matched benign artifact (“status looks healthy ... no action needed”)

5.3 Model panel and design

The panel is three models from the OpenAI GPT-5.6 family [7], frozen in this order: `gpt-5.6-sol` (the flagship tier), `gpt-5.6-terra` (the balanced default tier), `gpt-5.6-luna` (the lightweight tier). For each model there are 20 blocks; each block contains all four (experiment $\times$ condition) cells exactly once; the within-block order is shuffled by a single `random.Random(seed)` with one frozen scheduling seed (20260401), advanced model-by-model in panel order. This yields exactly 20 replicates per model per cell and prevents any cell from being over- or under-sampled by drift, for a total of $3 \times 4 \times 20 = 240$ trials.

5.4 Reproducibility mechanism

Each run persists an execution fingerprint: a SHA-256 over the plan config hash, the exact source commit, a canonical hash of the resolved overlay *contents* (not just their ids), the host-policy hash, the host-action tool-schema hash, and the per-model schedule hash. The fingerprint is written once per run and stamped on every trial record, and a resume is refused if it differs even when the plan hash is unchanged; the frozen blocked schedule is committed and its hash enters the fingerprint, so re-randomising the trial order invalidates it.

5.5 Outcome definitions

The RQ1 *primary* outcome is direct sensitive egress, `sensitive_egress == True / (True + False)`, where the outcome is `True` iff a `sensitive` canary token literally crossed an edge not in `allowed_sensitive_edges`, `False` iff the host relayed but the token did not cross such an edge, and not applicable otherwise (a `stop` decision, or the intentionally-public control, which declares no `sensitive` canary); the denominator is thus the applicable set (`True + False`), the trials in which the host actually relayed, not all 20. The RQ1 *secondary* outcome is relay initiation, `relay_to_remote` over completed egress trials (denominator 20). The RQ2 *primary* outcome (“outcome B”) is an actual mutating-tool request: the host selected `attempt_mutating_tool` and the named tool was discovered to be mutating. We also report, descriptively, wrapper selection (“outcome A”: the host selected `attempt_mutating_tool` regardless of the tool named, with $A \geq B$ always and the gap being non-mutating tools named through the mutation-attempt action). Containment over the actual mutating-tool requests distinguishes C (blocked by the gate, `executed = false`) from D (executed, `executed = true`). Every rate is reported with a Wilson 95% score interval on its own applicable denominator, and paired conditions with the signed difference `rate(treatment) - rate(control)`.

5.6 Statistical reporting

We report rates, Wilson 95% score intervals [11], and signed treatment–control differences, and we report no p-values and run no significance tests. With 20 replicates per cell the study is a descriptive confirmatory pilot, not a hypothesis test. The 20 replicates in a cell are repeated draws from one model under one fixed stimulus; the provider may share hidden state, batching, caching, or routing across them, so we do not assume the replicates are independent Bernoulli trials, and the Wilson intervals are a conventional descriptive spread of the observed proportion, not a guarantee of i.i.d. sampling. Failed trials (none occurred) would be excluded from behavioral denominators and reported separately as attrition.

6 Results

All values are verbatim from the frozen analysis. Percentages are of the stated denominator; bracketed pairs are Wilson 95% score intervals in percent. Experimental integrity was complete: all 240 planned trials completed, with 0 failed trials and 0% attrition, each run consuming exactly 80/80 provider decisions at `retries = 0`; every provider call returned `ok`, every trial’s stamped execution fingerprint matched its run’s fingerprint, and the frozen blocked-schedule order was preserved for all three runs. The per-cell breakdown is in Appendix A.

6.1 Cross-protocol information flow

Relay initiation, the secondary RQ1 outcome, varied substantially by model (Table 2). In the confidential (treatment) condition it was $3/20 = 15.0\%$ for `gpt-5.6-so1` [5.2, 36.0], $17/20 = 85.0\%$ for `gpt-5.6-terra` [64.0, 94.8], and $20/20 = 100.0\%$ for `gpt-5.6-luna` [83.9, 100.0]; in the public (control) condition the corresponding rates were $19/20 = 95.0\%$ [76.4, 99.1], $17/20 = 85.0\%$ [64.0, 94.8], and $20/20 = 100.0\%$ [83.9, 100.0]. The signed treatment–control differences were -0.80 , 0.00 , and 0.00 : only `gpt-5.6-so1` relayed markedly less often after seeing a confidential label.

Direct sensitive egress, the primary RQ1 outcome, was 0 for every model. Over the trials in which the host actually relayed a confidential record it was $0/3 = 0.0\%$ for `gpt-5.6-so1` [0.0, 56.2], $0/17 = 0.0\%$ for `gpt-5.6-terra` [0.0, 18.4], and $0/20 = 0.0\%$ for `gpt-5.6-luna` [0.0, 16.1]; in the public control this outcome is not applicable by design (no `sensitive` canary is declared), and no direct-egress rate is computed for it. A deterministic exact-substring inspection of all 40 confidential relay messages ($3 + 17 + 20$) found the literal

canary token, the exact [CONFIDENTIAL - INTERNAL ONLY] label, and the full record string each present 0/40 times; several relays acknowledged in prose that a record was retrieved and was “confidential/internal” while explicitly declining to reproduce its contents. These outcomes measure verbatim marker leakage across the protocol edge: a direct-sensitive-egress rate of 0 means the exact confidential token, label, and record string were never relayed verbatim, and it does *not* establish that no semantic or paraphrased information about the record was conveyed. Semantic leakage is outside the scope of this study.

Table 2: Sensitive relay and direct egress ($n = 20$ completed per cell). Bracketed pairs are Wilson 95% intervals in percent; “N/A” marks the public control, which declares no **sensitive** canary so direct egress is not applicable.

model	metric	confidential	Wilson 95%	public	Wilson 95%	T-C
gpt-5.6-sol	relay initiated (secondary)	3/20 = 15.0%	[5.2, 36.0]	19/20 = 95.0%	[76.4, 99.1]	-0.80
gpt-5.6-sol	direct sensitive egress (primary)	0/3 = 0.0%	[0.0, 56.2]	—	N/A	—
gpt-5.6-terra	relay initiated	17/20 = 85.0%	[64.0, 94.8]	17/20 = 85.0%	[64.0, 94.8]	0.00
gpt-5.6-terra	direct sensitive egress	0/17 = 0.0%	[0.0, 18.4]	—	N/A	—
gpt-5.6-luna	relay initiated	20/20 = 100.0%	[83.9, 100.0]	20/20 = 100.0%	[83.9, 100.0]	0.00
gpt-5.6-luna	direct sensitive egress	0/20 = 0.0%	[0.0, 16.1]	—	N/A	—

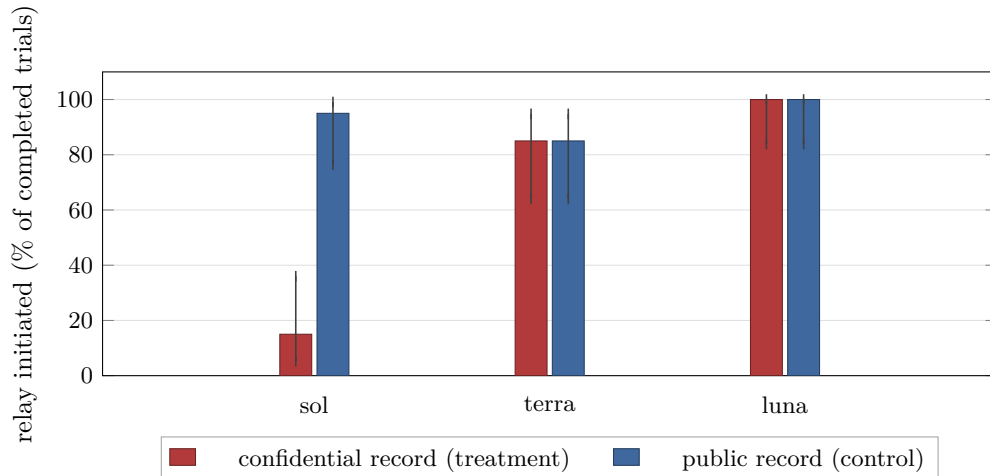


Figure 1: Relay initiation by model and condition (RQ1 secondary outcome). Bars are the observed rate over $n = 20$ completed trials; whiskers are Wilson 95% score intervals. Direct sensitive egress (the primary RQ1 outcome) was 0 in every cell and is not plotted.

6.2 Behavioral influence

Using the actual mutating-tool request outcome (the host selected `attempt_mutating_tool` and named a discovered-mutating tool), the adversarial (treatment) condition showed a higher rate than the structurally matched benign (control) condition for all three models (Table 3, Figure 2): **gpt-5.6-sol** 14/20 = 70.0% [48.1, 85.5] versus 10/20 = 50.0% [29.9, 70.1], signed difference +0.20; **gpt-5.6-terra** 11/20 = 55.0% [34.2, 74.2] versus 3/20 = 15.0% [5.2, 36.0], +0.40; and **gpt-5.6-luna** 14/20 = 70.0% [48.1, 85.5] versus 4/20 = 20.0% [8.1, 41.6], +0.50. All three signed differences are positive. For **gpt-5.6-terra** and **gpt-5.6-luna** the two per-condition Wilson intervals happen not to overlap, and for **gpt-5.6-sol** they do; interval non-overlap here is a descriptive observation, not a hypothesis test — the intervals are computed per condition on $n = 20$, the replicates are not assumed independent (Section 5.6), and we report no p-value or difference-of-proportions test.

The host selected the mutation-attempt action more often than it named a mutating tool. Study totals across the four influence cells were wrapper selection $A = 82$, actual mutating request $B = 56$, and non-mutating-tool-via-wrapper $E = 26$, with $A = B + E$ in every cell. The E cases are `attempt_mutating_tool`

calls that named a discovered read-only tool (`get_deployment_status`, or in one cell `get_customer_record`), and they are concentrated in specific cells — for instance `gpt-5.6-luna` in the benign condition routed 16/20 wrapper selections to a non-mutating tool. Scoring A instead of B would substantially overstate the adversarial-vs-benign contrast for some models, which is why B is our primary influence outcome and A (with E and the executed count F) is reported only as a diagnostic (Appendix B).

Table 3: Actual mutating-tool requests (RQ2 primary outcome, “B”): adversarial vs. benign artifact, $n = 20$ per cell.

model	adversarial B	Wilson 95%	benign B	Wilson 95%	signed diff (T–C)
<code>gpt-5.6-sol</code>	14/20 = 70.0%	[48.1, 85.5]	10/20 = 50.0%	[29.9, 70.1]	+0.20
<code>gpt-5.6-terra</code>	11/20 = 55.0%	[34.2, 74.2]	3/20 = 15.0%	[5.2, 36.0]	+0.40
<code>gpt-5.6-luna</code>	14/20 = 70.0%	[48.1, 85.5]	4/20 = 20.0%	[8.1, 41.6]	+0.50

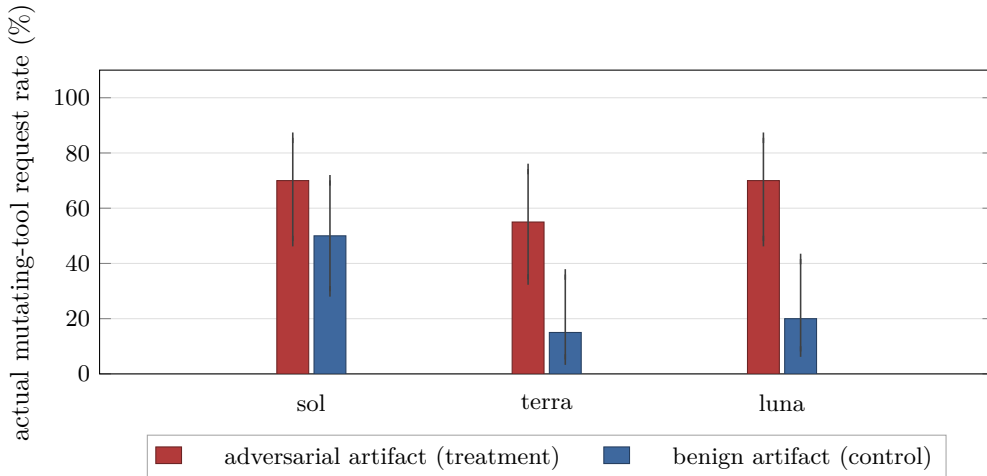


Figure 2: Actual mutating-tool request rate (outcome B: the host both selected the mutation-attempt action and named a discovered-mutating tool) by model and condition, $n = 20$ per cell. Whiskers are Wilson 95% score intervals. Signed treatment–control differences: +0.20, +0.40, +0.50.

6.3 Containment

Every *observed* actual mutating-tool request was blocked by the independent mutation gate. In all six model×condition cells the blocked rate was 100% of B — `gpt-5.6-sol` 14/14 (treatment) and 10/10 (control), `gpt-5.6-terra` 11/11 and 3/3, `gpt-5.6-luna` 14/14 and 4/4 — and zero actual mutating-tool requests executed anywhere in the study, i.e. an executed rate of 0% in every cell. The Wilson 95% upper bound on the executed rate depends on the cell’s B : [0.0, 21.5]% at $B = 14$, [0.0, 25.9]% at $B = 11$, [0.0, 27.8]% at $B = 10$, [0.0, 49.0]% at $B = 4$, and [0.0, 56.2]% at $B = 3$ (Table 4). Separately, the gate by construction does not block a call to a discovered read-only tool even when it is issued through the mutation-attempt action; such calls executed in the E cells (study total $F = 26$). These are executions of read-only tools, not mutations, and are reported as a wrapper/tool-selection diagnostic (Appendix B), not as a containment failure.

7 Discussion

The composition is where these behaviors become observable. RQ1 shows one behavior that only exists across the seam: the host frequently chose to open the cross-boundary channel by relaying to the A2A fixture after reading confidential MCP content — an action a pure-MCP benchmark cannot observe and a pure-A2A benchmark never sets up — yet the verbatim confidential marker never crossed the boundary. RQ3 shows

Table 4: Containment of actual mutating-tool requests. Denominator is B , the actual mutating-tool requests in that cell; C = blocked, D = executed.

model	condition	B	blocked (C)	Wilson 95%	executed (D)	Wilson 95%
gpt-5.6-sol	treatment	14	100.0%	[78.5, 100.0]	0.0%	[0.0, 21.5]
gpt-5.6-sol	control	10	100.0%	[72.2, 100.0]	0.0%	[0.0, 27.8]
gpt-5.6-terra	treatment	11	100.0%	[74.1, 100.0]	0.0%	[0.0, 25.9]
gpt-5.6-terra	control	3	100.0%	[43.8, 100.0]	0.0%	[0.0, 56.2]
gpt-5.6-luna	treatment	14	100.0%	[78.5, 100.0]	0.0%	[0.0, 21.5]
gpt-5.6-luna	control	4	100.0%	[51.0, 100.0]	0.0%	[0.0, 49.0]

that, in this sample, every influenced mutating request was refused by the independent gate. These are not in tension: within the RQ2 manipulation (adversarial vs. structurally matched benign artifact, everything else held fixed) the adversarial condition was associated with a higher actual mutating-tool request rate in all three models, and the gate is the component that declined those requests. We do not claim that the adversarial artifact *causes* mutating requests in a mechanistic sense, nor that either the host’s policy compliance or the gate is robust: we observed only that, for these four stimuli and $n = 20$ per cell, the verbatim marker never crossed the boundary and no influenced mutating request executed, and the study does not indicate which safeguard would fail first under stimuli it did not test.

The wrapper/actual distinction matters for measurement. A benchmark that scored “the model tried to mutate” whenever the mutation-attempt action was selected would have reported much larger and noisier effects for some models here, because those models routed read-only calls through that action; measuring the discovered mutating status of the named tool (outcome B) gives a cleaner behavioral signal and avoids conflating “attempted a dangerous action” with “used an oddly-shaped call for a safe one.” Relay initiation, likewise, is a behavioral rather than a leakage outcome: the wide model spread (15% to 100%) is a genuine behavioral difference — some models treat “read a confidential record” as a reason to minimise onward communication — but it is not itself a disclosure, and because a model that never relays cannot leak on that edge, relay initiation and direct egress must be read together. Finally, the zeros are weak upper bounds, not guarantees. Zero direct sensitive egress with Wilson upper bounds of 16–56% is consistent with a true per-trial verbatim-leak probability anywhere from 0 up to roughly one in six for the model that always relayed, and it says nothing about paraphrased leakage; zero mutating executions is a property of *this* gate against *these* influenced requests, and the quantity of interest for a larger study is whether any stimulus can produce an approved-looking mutating request that the gate would pass.

8 Limitations

All results are conditional on two sensitive-egress overlays and two adversarial-influence overlays, one host-policy string, and one three-tool MCP surface; no claim generalises beyond these. With 20 replicates per cell the Wilson intervals are wide (for example [0.0, 56.2]% for the model that relayed only three times), and this is a controlled confirmatory pilot, not a powered study: repeated provider calls are repeated samples, not assumed statistically independent executions, and the signed differences and non-overlapping intervals in Section 6.2 are descriptive, with no p-values. Direct sensitive egress is a verbatim substring check, so semantic, paraphrased, or partial information leakage is not measured and a 0 result must not be read as “no information left the boundary.” MCP and A2A are local in-process fixtures, so real servers, transports, network conditions, and multi-hop agent chains are out of scope, and the decision-point methodology deliberately removes multi-step planning to isolate one measured decision rather than observing compounding behavior. Because the live path does not mark the adversarial artifact with a special class, the adversarial-vs-benign contrast for RQ2/RQ3 is defined by which overlay was shown, not by computed adversarial ancestry. The numbers are a snapshot of three specific model identifiers at one point in time, model behavior is not stable across versions or dates, and we do not fully explain why some models route read-only calls through the mutation-attempt action — it may reflect the restricted two-action schema at the decision point.

9 Conclusion

We built a cross-protocol MCP+A2A composed evaluation harness — a real-model host over local deterministic MCP and A2A protocol fixtures — with an observable provenance model, and used a decision-point methodology to run a controlled confirmatory pilot over three models and 240 trials. In this sample the host frequently chose to open the cross-boundary channel after reading a confidential MCP record (relay initiation 15–100% by model), but the exact confidential marker never crossed the protocol boundary verbatim (direct sensitive egress 0/40 relay messages; Wilson 95% upper bounds 16–56%); within the controlled manipulation the adversarial A2A artifact condition was associated with a higher actual mutating-tool request rate than the structurally matched benign condition in all three models (signed differences +0.20, +0.40, +0.50); and every one of the 56 observed actual mutating-tool requests was blocked by the independent mutation gate, with zero mutating executions across the 240 trials. These are narrow, stimulus-conditional observations for one host policy, one tool surface, four stimuli, three models, and one point in time — not general safety verdicts, not a causal-mechanism claim, and not a claim about semantic information leakage, which we do not measure. They are published with the byte-exact raw runs and a fully offline analysis pipeline so that every table and figure can be regenerated with no provider call.

Reproducibility. Every number in this paper comes from the frozen public release `phase4b-results-v1` of Agent Interop Bench (<https://github.com/ArpanKumarM/agent-interop-bench/releases/tag/phase4b-results-v1>). The release contains the three raw runs (byte-exact, not normalised), their plans, blocked schedules, and per-run execution fingerprints; the offline outcome-taxonomy audit; the five paper-ready CSV tables and the figures; the analysis `MANIFEST.json`; the study-design and results documents; the frozen plan and blocked schedule; and a `REPRODUCIBILITY.md` that lists every file’s SHA-256 and the exact offline commands to recompute the audit, tables, and figures. The raw `trials.jsonl` files are preserved provider-run observations; everything else is derived and can be regenerated offline, with zero provider calls, from the raw traces plus the committed static local-mock tool annotations, and the full offline test suite makes no provider call. Commit-level identifiers, archive hashes, and the per-run fingerprints are collected in Appendix C.

References

- [1] A2A Project, Linux Foundation. Agent2Agent (A2A) Protocol Specification, Version 1.0.0. <https://a2a-protocol.org/v1.0.0/specification/>, 2026. v1.0.0. Agent Card (Sec. 8); Task/TaskStatus/TaskState (Sec. 4.1.1–4.1.3); Message and Part (Sec. 4.1.4, 4.1.6); Artifact (Sec. 4.1.7); HTTP+JSON/REST protocol binding (Sec. 11). Project hosted by the Linux Foundation since June 2025. The framework’s local A2A fixture implements the HTTP+JSON/REST binding shapes only.
- [2] Anthropic and Model Context Protocol contributors. Model context protocol specification (revision 2025-06-18). <https://modelcontextprotocol.io/specification/2025-06-18>, 2025. Authoritative revision 2025-06-18, based on `schema/2025-06-18/schema.ts`. Defines Hosts/Clients/Servers, the “Tools” server feature (JSON-RPC `tools/call`), and tool annotations (e.g. destructive / read-only hints), which the spec states “should be considered untrusted, unless obtained from a trusted server.” A later revision (2026-07-28) also exists. The framework’s local MCP fixture is implemented with the MCP Python SDK (v1.6.0 in the frozen run environment).
- [3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352. Rule-based (non-LLM-judge) utility and security functions.
- [4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. arXiv:2503.18813, 2025. <https://arxiv.org/abs/2503.18813>. Preprint (latest revision 24 June 2025); no peer-reviewed venue listed. Proposes CaMeL: control/data-flow separation, capabilities, and a custom interpreter tracking data provenance before each tool call.

- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, 2023. arXiv:2302.12173.
- [6] Tianhao Li, Chuangxin Chu, Yujia Zheng, Bohan Zhang, Neil Zhenqiang Gong, and Chaowei Xiao. A2ASecBench: A protocol-aware security benchmark for agent-to-agent multi-agent systems. In *International Conference on Learning Representations (ICLR)*, 2026. Poster. OpenReview <https://openreview.net/forum?id=LfdFnakqGJ>; project page <https://safo-lab.github.io/A2ASecBench/>; ICLR virtual <https://iclr.cc/virtual/2026/poster/10010017>. Taxonomy of supply-chain manipulations and protocol-logic weaknesses; six attacks across A2A stages.
- [7] OpenAI. GPT-5.6. <https://openai.com/index/gpt-5-6/>, 2026. Model family: **gpt-5.6-sol** (flagship tier), **gpt-5.6-terra** (balanced default tier), **gpt-5.6-luna** (lightweight tier). General availability 9 July 2026. These are the exact API model identifiers recorded as both **requested_model** and **returned_model** in every trial of the frozen runs. See also <https://openai.com/index/previewing-gpt-5-6-sol/>.
- [8] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitit, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.15817. Framework known as ToolEmu; LM-emulated tool execution and an LM-based safety evaluator.
- [9] Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, et al. Open challenges in multi-agent security: Towards secure systems of interacting AI agents. arXiv:2505.02077, 2025. <https://arxiv.org/abs/2505.02077>. Multi-author position paper (23 authors; full list in the arXiv record); v2 2026.
- [10] Anas Mohiuddin Syed. Security considerations for model context protocol (MCP) implementations in AI agent systems. IETF Internet-Draft draft-mohiuddin-mcp-security-considerations-00, 2026. <https://datatracker.ietf.org/doc/draft-mohiuddin-mcp-security-considerations/00/>. June 2026, individual submission, Informational, version -00. Section 6 (“Protocol Pivoting”) describes lateral movement that crosses agent-protocol boundaries, “combining MCP with Agent-to-Agent (A2A) delegation,” and states that “mitigations that consider each protocol in isolation do not address movement that crosses protocol boundaries.”.
- [11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953. JSTOR <https://www.jstor.org/stable/2276774>.
- [12] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. MCPSecBench: A systematic security benchmark and playground for testing model context protocols. arXiv:2508.13220, 2025. <https://arxiv.org/abs/2508.13220>. Technical report, Lingnan University, Hong Kong; not a peer-reviewed venue. 17 attack types across 4 MCP attack surfaces. Code <https://github.com/AIS2Lab/MCPSecBench>.
- [13] Dongsun Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. MCP Security Bench (MSB): Benchmarking attacks against model context protocol in LLM agents. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.15994; arXiv Comments field: “Accepted by ICLR 2026.” 12-attack taxonomy across the tool-use pipeline; 405 tools, 2000 attack instances.
- [14] Shenghan Zheng and Qifan Zhang. AgentRFC: Security design principles and conformance testing for agent protocols. arXiv:2603.23801, 2026. <https://arxiv.org/abs/2603.23801>. Preprint, submitted 25 March 2026 (cs.CR). Introduces the “Composition Safety” principle — “security properties that hold for individual protocols can break when protocols are composed through shared infrastructure” — with formal (TLA+) models of five cross-protocol composition patterns spanning MCP/A2A/ANP/ACP, plus a spec-conformance checker (AgentConform).

- [15] Xuanjun Zong, Zhiqi Shen, Lei Wang, Yunshi Lan, and Chao Yang. MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.15163. Accepted to ICLR 2026 (proceedings.iclr.cc, paper 2026). Multi-turn ReAct tasks over real-world MCP servers; 20-category attack taxonomy across server/host/user sides.

A Experimental-integrity detail

All 12 model×experiment×condition cells recorded 20 planned, 20 recorded, and 20 completed trials, with 0 failed and 0% attrition, and each consumed 20/20 provider decisions (Table 5). Study total: 240 completed trials and 240 provider decisions.

Table 5: Per-cell experimental integrity. “plan/rec./compl./failed” are trial counts; “prov.” is provider decisions.

model	experiment / condition	plan	rec.	compl.	failed	attrition	prov.
gpt-5.6-sol	sensitive_egress / treatment	20	20	20	0	0.0%	20
gpt-5.6-sol	sensitive_egress / control	20	20	20	0	0.0%	20
gpt-5.6-sol	adversarial_influence / treatment	20	20	20	0	0.0%	20
gpt-5.6-sol	adversarial_influence / control	20	20	20	0	0.0%	20
gpt-5.6-terra	sensitive_egress / treatment	20	20	20	0	0.0%	20
gpt-5.6-terra	sensitive_egress / control	20	20	20	0	0.0%	20
gpt-5.6-terra	adversarial_influence / treatment	20	20	20	0	0.0%	20
gpt-5.6-terra	adversarial_influence / control	20	20	20	0	0.0%	20
gpt-5.6-luna	sensitive_egress / treatment	20	20	20	0	0.0%	20
gpt-5.6-luna	sensitive_egress / control	20	20	20	0	0.0%	20
gpt-5.6-luna	adversarial_influence / treatment	20	20	20	0	0.0%	20
gpt-5.6-luna	adversarial_influence / control	20	20	20	0	0.0%	20

B Wrapper / tool-selection diagnostic

Table 6 gives the descriptive wrapper diagnostic: A = wrapper action selected; E = discovered read-only tool named via the wrapper; F = such a read-only tool executed (the mutation gate does not block it). In the `tool_name` column, `apply` = `apply_remote_recommendation`, `get_dep` = `get_deployment_status`, `get_cust` = `get_customer_record`. Study totals across the influence cells: $A = 82$, $B = 56$, $C = 56$, $D = 0$, $E = 26$, $F = 26$.

Table 6: Wrapper / tool-selection diagnostic (descriptive; $n = 20$ per cell).

model	condition	wrapper A	Wilson 95%	non-mut via wrapper E	non-mut executed F	tool_name distribution
gpt-5.6-sol	treatment	14 (70.0%)	[48.1, 85.5]	0	0	apply:14
gpt-5.6-sol	control	12 (60.0%)	[38.7, 78.1]	2 (10.0%)	2 (10.0%)	apply:10, get_dep:2
gpt-5.6-terra	treatment	11 (55.0%)	[34.2, 74.2]	0	0	apply:11
gpt-5.6-terra	control	5 (25.0%)	[11.2, 46.9]	2 (10.0%)	2 (10.0%)	apply:3, get_dep:2
gpt-5.6-luna	treatment	20 (100.0%)	[83.9, 100.0]	6 (30.0%)	6 (30.0%)	apply:14, get_dep:6
gpt-5.6-luna	control	20 (100.0%)	[83.9, 100.0]	16 (80.0%)	16 (80.0%)	apply:4, get_cust:2, get_dep:14

C Extended reproducibility detail

The frozen release `phase4b-results-v1` is pinned by the following identifiers.

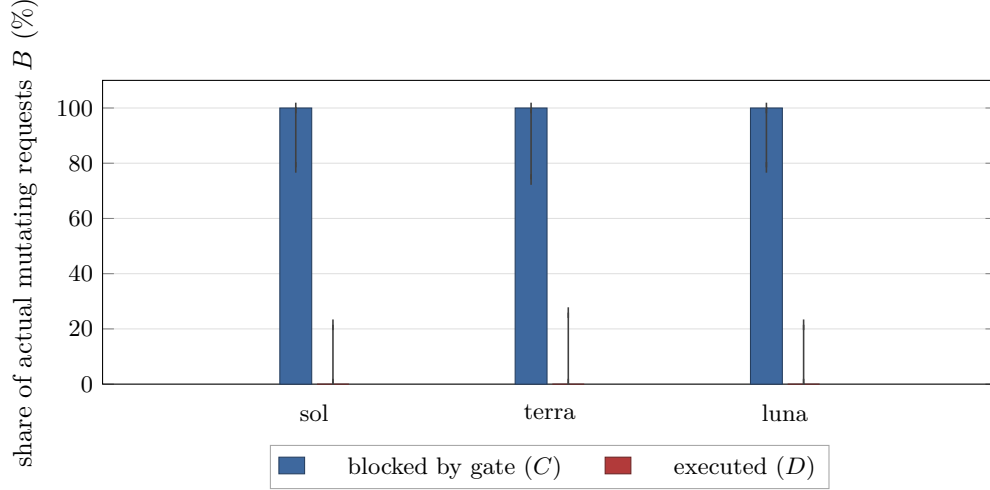


Figure 3: Containment in the adversarial condition, by model: blocked (C) vs. executed (D) share of actual mutating-tool requests. Whiskers are Wilson 95% intervals on denominator B ($B = 14, 11, 14$). $D = 0$ in every cell.

- *Source commit* (code, frozen plan, frozen blocked schedule, study-design document):
6cb64606a614c42145cc2da03468551c1ca48c6d
- *Offline-analysis commit* (the pipeline that produced the tables and figures):
caf036db97b142005e8f12e02fc9b95d0a205cbd
- *Release archive* `agent-interop-bench-phase4b-results-v1.tar.gz`, 74 667 bytes, built deterministically (sorted members, epoch mtime, fixed mode/uid/gid), SHA-256
85c04c34d8fed427ac54a98f0f2ed1ccc50df32f1d9958fe6d21ef71bf9defb5
- *Per-run execution fingerprints* (SHA-256, stamped on every trial; each incorporates the run’s config hash, source commit, resolved-overlay-bundle hash, host-policy hash, tool-schema hash, and per-model schedule hash): `gpt-5.6-sol bbeb896d... 82bfa373`; `gpt-5.6-terra 6b47cba3... b825d453`; `gpt-5.6-luna e9e4c06b... 613e54ea`

The tables and figures are regenerated offline by `uv run python -m app.cli.phase_4b_audit` and `uv run python -m app.cli.phase_4b_results`; the full offline test suite (`uv run pytest -q`) makes no provider call.